

Questionário (ref. Aulas 10 a 16)

Métricas p/ Modelos de Classificação

1) O que a **acurácia** mede em um modelo de classificação?

*Essa é uma métrica básica. A acurácia responde: de tudo que o modelo previu, quantas previsões estão corretas? Ou seja, mede o acerto geral do modelo. O cálculo é: **Acurácia** = $(VP + VN) / (VP + VN + FP + FN)$.*

Onde: VP = Verdadeiros Positivos, VN = Verdadeiros Negativos, FP = Falsos Positivos; e FN = Falsos Negativos.

2) O que a **precisão** mede em um modelo de classificação?

*Imagine que o modelo aponta algumas pessoas dizendo “essas vão comprar”. A precisão responde: dessas que eu aponte, quantas realmente compraram? Ou seja, mede o quão confiáveis são as “previsões positivas” do modelo. O cálculo é: **Precisão** = $VP / (VP + FP)$. Note que a ideia aqui é: “Quando o modelo tenta prever positivo, o quanto ele acerta?”*

Vale notar que podemos aumentar a precisão artificialmente criando modelos conservadores. Ou seja, que só fazem previsões positivas “na certeza”. Porém, isso nem sempre é bom, pois, dessa forma, vários positivos não serão capturados. Por isso, outras métricas são necessárias.

3) O que o **recall** mede e qual a principal diferença em relação à precisão?

*Agora pense em todas as pessoas que realmente compraram. O recall responde: de todas essas, quantas o modelo conseguiu encontrar? O cálculo é: **Recall** = $VP / (VP + FN)$. Note que a fórmula é parecida com a da precisão, mas trocando o falso positivo pelo falso negativo. Isso ocorre porque o recall mede quantos **positivos reais** o modelo encontrou. O falso negativo representa justamente “o positivo que o modelo deixou prever”. Note que modelos excessivamente conservadores (que evitam prever positivos) podem apresentar alta precisão, mas baixo recall.*

4) O que o **F1-score** mede conceitualmente e por que ele é importante?

O F1-score mede o equilíbrio entre precisão e recall. Ele é importante porque evita avaliações enganosas em que o modelo apresenta desempenho muito alto em apenas uma dessas métricas e muito baixo na outra. Como utiliza a média harmônica, o F1-score tende a cair quando existe grande desequilíbrio entre precisão e recall. O cálculo é:

$$F1 = 2 \times (Precisão \times Recall) / (Precisão + Recall).$$

5) Por que o F1-score usa média harmônica e não média simples?

Porque ele “pune” quando um dos lados está ruim. Imagine um aluno com nota 10 em uma prova e 1 na outra. A média simples seria 5,5, mas a média harmônica seria aproximadamente 1,82, mostrando como um valor

Questionário (ref. Aulas 10 a 16)

muito baixo derruba fortemente o resultado final. No F1-score acontece algo parecido: se precisão ou recall forem baixos, F1 também cai bastante.

Capítulo V – Especificação de modelos

KNN - Características

6) O que é o KNN e em que tipo de aprendizado ele se enquadra?

É um algoritmo de aprendizado supervisionado utilizado tanto para classificação quanto para regressão. Seu funcionamento é baseado na ideia de que registros semelhantes tendem a apresentar respostas semelhantes. Para realizar uma previsão, o algoritmo localiza os K vizinhos mais próximos no espaço de atributos e utiliza essas informações para decidir o resultado. Em problemas de classificação, a decisão normalmente é feita por votação entre os vizinhos. Já em problemas de regressão, a previsão costuma ser realizada pela média dos valores dos vizinhos.

7) KNN pode ser usado para regressão? Como funciona nesse caso?

Sim, o KNN pode ser usado para regressão. Nesse caso, em vez de prever uma classe, ele prevê um valor numérico contínuo. O funcionamento é semelhante ao da classificação: o algoritmo calcula a distância entre o novo ponto e todos os dados de treino, seleciona os K vizinhos mais próximos e, em seguida, calcula a média dos valores da variável-alvo desses vizinhos. Em algumas variações, pode-se usar uma média ponderada pela distância (dando mais peso aos vizinhos mais “próximos” entre os próximos”).

8) KNN é modelo paramétrico ou não paramétrico? Por quê?

*Parâmetros **são valores aprendidos durante o treinamento do modelo**, como os pesos ‘w’ e o intercepto ‘b’ da regressão linear, por exemplo. O KNN é não paramétrico porque ele não aprende uma função com parâmetros fixos. Em vez disso, ele “memoriza” os dados e usa esses próprios dados na hora da predição. Por isso a complexidade cresce com o dataset: Quanto mais dados mais demorada fica a previsão pois todos eles são recalculados no momento em que a previsão está sendo feita.*

9) Qual efeito de uma escolha de K pequeno no KNN?

Quando K é pequeno, o KNN considera poucos vizinhos, tornando a decisão mais local. Isso aumenta a sensibilidade a ruídos e variações individuais dos dados, resultando em uma fronteira de decisão mais irregular (muita variabilidade) e adaptada ao conjunto de treino (baixo viés). Nesse caso, o modelo apresenta baixo viés e alta variância, podendo capturar padrões muito específicos, inclusive ruído, o que pode levar a overfitting.

K muito pequeno → baixo viés → alta variância → overfitting

Questionário (ref. Aulas 10 a 16)

10) Qual efeito de uma escolha de K grande no KNN?

Quando K é grande, o KNN considera muitos vizinhos, tornando a decisão mais global. Isso reduz a sensibilidade a ruídos e variações individuais dos dados, resultando em uma fronteira de decisão mais suave e estável. Porém se K for muito grande incorremos no problema contrário: o modelo apresenta alto viés e baixa variância, podendo deixar de capturar padrões locais relevantes, o que pode levar a underfitting.

K muito grande → alto viés → baixa variação → underfitting

11) O que são algoritmos “lazy learner”?

São algoritmos que não constroem explicitamente um modelo generalizado durante a etapa de treinamento. Em vez disso, armazenam os dados de treino e adiam grande parte do processamento para o momento da predição. O KNN é considerado um lazy learner porque, ao receber um novo registro, compara esse ponto com os exemplos armazenados e toma a decisão com base nos vizinhos mais próximos. Assim, o custo computacional tende a ser baixo durante o treinamento, mas mais elevado na fase de predição.

12) O que é “fronteira de decisão” no KNN e como ela se comporta?

A fronteira de decisão é a separação entre regiões do espaço de atributos associadas a diferentes classes. No KNN, essa fronteira não é definida por uma função matemática explícita, mas emerge da distribuição dos dados. Com K pequeno, a fronteira tende a ser muito irregular e adaptada aos dados, podendo seguir até ruídos (overfitting). Com k maior, a fronteira se torna mais suave e generalizada, refletindo um comportamento mais global dos dados.

KNN - Modelagem e Distâncias

13) Por que a escolha da métrica de distância é crítica no KNN?

A métrica de distância é crítica porque o KNN toma decisões com base na proximidade entre os registros. Assim, a forma como a distância é calculada influencia diretamente quais exemplos serão considerados vizinhos. Se a métrica não representar corretamente a similaridade entre os dados, o algoritmo poderá selecionar vizinhos inadequados e produzir classificações incorretas. Além disso, diferentes métricas podem alterar significativamente o comportamento do modelo dependendo dos dados.

14) Cite duas métricas de distância e quando usá-las.

Dois métricas comuns são a distância euclidiana, usada quando as variáveis são numéricas e contínuas, e a distância de Manhattan, usada quando se deseja maior robustez a outliers ou quando as variáveis

Questionário (ref. Aulas 10 a 16)

representam diferenças absolutas (ex.: dados com muitas dimensões ou distribuição não uniforme).

15) Qual a diferença entre distância Euclidiana e Manhattan?

A distância Euclidiana mede a distância em linha reta entre dois pontos no espaço vetorial, enquanto a distância Manhattan mede a soma das diferenças absolutas entre as variáveis, como se o deslocamento ocorresse por “quadras” em uma cidade. A Euclidiana é mais sensível a grandes diferenças e outliers; a Manhattan tende a ser mais robusta nesses casos.

Por exemplo, considere os pontos $A=(1,1)$ e $B=(4,5)$.

Na distância Euclidiana, calculamos a distância em linha reta: $\sqrt{(4-1)^2 + (5-1)^2} = \sqrt{9+16} = \sqrt{25} = 5$; na Manhattan, somamos as diferenças absolutas: $|4-1| + |5-1| = 3 + 4 = 7$.

16) A escolha da métrica de distância depende do tipo de problema e dos dados? Justifique com exemplos.

Sim, a escolha da métrica de distância depende diretamente do tipo de problema e da natureza dos dados, pois ela define o que significa “ser próximo” ou “ser similar”. Diferentes métricas capturam diferentes noções de similaridade. Por exemplo, a distância euclidiana é adequada quando o interesse está na distância absoluta entre os pontos no espaço vetorial (magnitude das diferenças), enquanto a similaridade de cosseno é mais apropriada quando o objetivo é comparar o alinhamento ou o padrão entre vetores, independentemente do seu tamanho ou magnitude. Assim, a escolha da métrica deve estar alinhada ao significado dos dados e ao objetivo da análise, pois uma métrica inadequada pode levar à seleção incorreta de vizinhos e comprometer o desempenho do modelo.

17) Por que é necessário o escalonamento no KNN?

O escalonamento é necessário porque o KNN depende diretamente de distâncias. Como essas distâncias são influenciadas pela escala das variáveis, é preciso colocá-las em uma mesma escala para que nenhuma variável domine indevidamente o cálculo. Entre as técnicas de escalonamento mais comuns estão a normalização e a padronização.

18) O que acontece se variáveis estão em escalas muito diferentes?

Se as variáveis estiverem em escalas muito diferentes, aquelas com valores maiores terão maior influência no cálculo da distância, fazendo com que o modelo ignore outras variáveis potencialmente relevantes e prejudique a qualidade das previsões. Ou seja, o modelo tende a apresentar um desempenho pior.

Questionário (ref. Aulas 10 a 16)

KNN - Problemas e Limitações

19) Explique a “maldição da dimensionalidade” no contexto do KNN.
A maldição da dimensionalidade refere-se ao fato de que, à medida que o número de dimensões aumenta (mais variáveis/features), o espaço de dados cresce exponencialmente e os pontos tendem a ficar todos muito “distantes” uns dos outros. Como consequência, as diferenças entre distâncias “grandes” e “pequenas” diminuem (tudo fica “grande”), tornando difícil identificar quem realmente são os vizinhos mais próximos. Isso prejudica diretamente o KNN, pois ele depende exatamente dessa noção de “proximidade” para funcionar bem.

20) Por que o KNN tem alto custo na fase de predição?
O KNN possui alto custo na fase de predição porque não realiza um treinamento prévio para aprender um modelo compacto. Em vez disso, ele armazena todos os dados e, a cada nova predição, precisa calcular a distância entre o novo ponto e todos os exemplos do conjunto de treino. Esse processo é computacionalmente caro, especialmente em datasets grandes, pois cresce linearmente com o número de observações.

21) Como o tamanho do dataset impacta o desempenho do KNN?
O tamanho do dataset impacta o KNN de duas formas principais. Por um lado, mais dados podem melhorar a qualidade das predições, pois aumentam a representatividade do espaço e a chance de encontrar vizinhos realmente semelhantes. Por outro lado, quanto maior o dataset, maior será o custo computacional na predição, já que o algoritmo precisa comparar o novo ponto com todos os exemplos disponíveis. Assim, há um trade-off entre desempenho e eficiência.

22) KNN é sensível a ruído/outliers? Por quê?
Sim, o KNN é sensível a ruído e outliers porque suas decisões são baseadas diretamente nos vizinhos mais próximos. Se um ponto ruidoso ou um outlier estiver próximo do novo ponto, ele pode influenciar fortemente a predição, especialmente quando K é pequeno. Isso ocorre porque, com poucos vizinhos, cada ponto tem grande peso na decisão. Aumentar K pode reduzir esse efeito, pois dilui a influência de pontos isolados.

KNN - Boas Práticas

23) Como escolher o melhor valor de K ?
O melhor valor de k é escolhido de forma empírica, testando diferentes valores e avaliando o desempenho do modelo em dados de validação. A prática comum é utilizar validação cruzada para comparar métricas (ex.: acurácia, $F1$, MAE , $RMSE$) para diferentes valores de K e selecionar aquele

Questionário (ref. Aulas 10 a 16)

que apresenta melhor desempenho. Valores muito pequenos tendem a gerar alta variância (overfitting), enquanto valores muito grandes aumentam o viés (underfitting).

24) O que é validação cruzada no contexto do KNN?

Validação cruzada é uma técnica de avaliação em que o conjunto de dados é dividido em várias partes (folds). O modelo é treinado em parte dos dados e validado na parte restante, repetindo esse processo várias vezes com diferentes divisões. No contexto do KNN, ela pode ser usada para avaliar diferentes valores de K e métricas de distância, garantindo que o desempenho observado não dependa de uma única divisão dos dados.

25) Quando o KNN é uma boa escolha de modelo?

O KNN é uma boa escolha quando o dataset é relativamente pequeno ou médio, quando a relação entre variáveis é complexa e não linear, e quando a noção de similaridade (distância) faz sentido para o problema. Também é útil como baseline simples e em cenários onde a interpretabilidade local (baseada em vizinhos) é desejada.

26) Quando o KNN deve ser evitado?

O KNN deve ser evitado em datasets muito grandes, devido ao alto custo computacional na predição, e em problemas com alta dimensionalidade, onde a noção de distância perde significado (maldição da dimensionalidade). Também não é indicado quando há muito ruído ou quando as variáveis não estão bem escalonadas, pois isso pode distorcer a definição de vizinhança e prejudicar o desempenho do modelo.

Questionário (ref. Aulas 10 a 16)

Capítulo V – Especificação de modelos

Árvores de Decisão- Características

27) Como árvores de decisão realizam previsões?

As árvores realizam previsões por meio de uma estrutura hierárquica de decisões sequenciais baseadas nos dados. O modelo divide recursivamente o espaço de dados em regiões mais homogêneas, utilizando regras do tipo “se-então” (if/else). Cada condição direciona o registro para um novo ramo até chegar a uma folha, onde a previsão final é produzida.

28) O que significa dividir recursivamente o espaço de dados?

Significa separar sucessivamente os dados em subconjuntos menores com base em regras de decisão. A cada divisão, o algoritmo continua subdividindo os grupos obtidos até atingir um critério de parada. A ideia é que, a cada divisão o nível de mistura dos dados diminua.

29) O que representa um nó em uma árvore?

Um nó representa um ponto de decisão da árvore, normalmente associado a uma condição sobre um atributo, como ‘idade < 30’ ou ‘peso > 50’. Em muitos casos, os nós também apresentam informações adicionais, como a regra de decisão utilizada, quantidade de amostras presentes no nó, distribuição das classes e métricas de impureza ou qualidade da divisão, como Gini ou entropia.

30) O que representa uma folha?

Uma folha representa a decisão final da árvore. Em classificação, normalmente corresponde à classe prevista. Em regressão, corresponde ao valor numérico previsto.

31) O que é um split em uma árvore de decisão?

Um split é uma divisão dos dados realizada com base em uma condição sobre um atributo. Por exemplo, a árvore pode criar uma regra como ‘peso > 150’ para separar os registros em dois grupos diferentes. O objetivo dessa divisão é reduzir a mistura entre as classes, criando ramos mais homogêneos e puros. Ou seja, após o split, a árvore tenta fazer com que cada ramo passe a concentrar mais registros de uma mesma classe, tornando os grupos mais homogêneos e puros.

32) Qual o objetivo de um split em uma árvore de decisão?

O objetivo do split é reduzir a mistura entre as classes e tornar os nós mais homogêneos. Por exemplo, se antes de uma divisão um nó possui registros muito misturados entre “maçã”, “laranja” e “banana”, a árvore tenta encontrar uma regra que concentre mais registros de uma mesma classe em cada ramo, aumentando a pureza dos nós.

Questionário (ref. Aulas 10 a 16)

33) Uma escola utilizou uma árvore de decisão para classificar alunos em três categorias de desempenho final: alto desempenho, desempenho médio e baixo desempenho, utilizando atributos como frequência, média das provas, participação e quantidade de exercícios entregues. Após percorrer a árvore, um novo registro de aluno chegou a um nó terminal contendo 3 alunos de alto desempenho, 16 de desempenho médio e 1 de baixo desempenho. Com base nesse cenário, responda:

a. Qual será a previsão produzida pela árvore para esse aluno?

A previsão será “desempenho médio”, pois essa é a classe majoritária presente no nó terminal.

b. O nó é totalmente puro?

O nó não é totalmente puro, pois ainda existe mistura entre as classes. Embora a maior parte dos registros pertença à categoria “desempenho médio”, ainda há alunos classificados como “alto desempenho” e “baixo desempenho” dentro do mesmo nó.

34) Uma empresa utilizou uma árvore de regressão para prever o valor gasto por clientes em compras futuras, utilizando atributos como renda, idade, frequência de compras e tempo de cadastro. Após percorrer a árvore, um novo registro de cliente chegou a um nó terminal contendo clientes com os seguintes valores de gasto: R\$ 900, R\$ 1100, R\$ 1000 e R\$ 1200. Com base nesse cenário, responda:

a. Qual será aproximadamente a previsão produzida pela árvore para esse cliente?

A previsão será aproximadamente R\$ 1050, correspondente à média dos valores presentes no nó terminal.

b. Os valores presentes no nó precisam ser totalmente iguais?

Não. Em árvores de regressão, o objetivo é reduzir a variabilidade dos valores dentro dos nós, tornando-os mais semelhantes entre si, mas não necessariamente idênticos.

35) Em uma árvore de decisão para classificação, um determinado nó possui a seguinte distribuição de classes: 19 registros da Classe A, 1 registro da Classe B e nenhum registro das demais classes. O que isso indica sobre a pureza do nó e sobre o valor esperado do índice Gini?

Esse nó possui alta pureza, pois quase todos os registros pertencem à mesma classe. Nesse caso, o índice Gini tende a assumir um valor próximo de zero, indicando baixa mistura entre as classes.

Questionário (ref. Aulas 10 a 16)

- 36) Em uma árvore de regressão, um determinado nó possui os seguintes valores da variável-alvo: 1000, 1010, 995 e 1005. O que isso indica sobre o erro quadrático médio (MSE) desse nó?

Isso indica que o nó possui baixa variabilidade entre os valores, pois todos estão próximos entre si. Consequentemente, o MSE tende a ser baixo, indicando um grupo mais homogêneo em termos numéricos.

- 37) Cite três dos principais hiperparâmetros utilizados em árvores de decisão.

*No caso da árvore, os principais hiperparâmetros têm como função primordial controlar o crescimento da árvore e ajudam a evitar overfitting e underfitting. Alguns dos principais hiperparâmetros são: **max_depth**: define a profundidade máxima da árvore; **min_samples_split**: define a quantidade mínima de amostras necessária para dividir um nó; **min_samples_leaf**: define a quantidade mínima de amostras permitida em uma folha. Esses hiperparâmetros controlam o crescimento da*

- 38) Por que árvores conseguem capturar relações não lineares?

Porque árvores de decisão realizam divisões sequenciais no espaço de dados por meio de regras condicionais ("se-então"), criando regiões complexas e não lineares sem necessidade de transformações explícitas nas variáveis.

- 39) Por que árvores são suscetíveis a overfitting?

Porque árvores de decisão realizam divisões sequenciais no espaço de dados por meio de regras condicionais ("se-então"), criando regiões complexas e não lineares. Assim, o modelo consegue representar padrões complexos sem precisar assumir relações lineares entre as variáveis.

- 40) O que acontece se a árvore crescer demais?

Se a árvore crescer demais, ela tende a se ajustar excessivamente aos dados de treino, capturando inclusive ruídos e variações aleatórias. Isso aumenta a variância do modelo e pode levar a overfitting, prejudicando a generalização em novos dados.

- 41) O que acontece se a árvore crescer pouco?

Se a árvore crescer pouco, ela pode não capturar padrões importantes dos dados, tornando o modelo excessivamente simples. Isso aumenta o viés e pode levar a underfitting, reduzindo a qualidade das previsões.

- 42) Qual o papel do parâmetro max_depth?

O parâmetro max_depth define a profundidade máxima da árvore, limitando quantos níveis de divisões ela pode criar. Seu principal objetivo é controlar a complexidade do modelo e evitar overfitting.

Questionário (ref. Aulas 10 a 16)

43) Para que serve `min_samples_split`?

O parâmetro `min_samples_split` define a quantidade mínima de amostras necessária para que um nó possa ser dividido. Isso impede que a árvore continue criando splits em grupos muito pequenos.

44) Para que serve `min_samples_leaf`?

O parâmetro `min_samples_leaf` define a quantidade mínima de amostras permitida em uma folha. Seu objetivo é evitar folhas muito pequenas e excessivamente específicas, ajudando a reduzir overfitting.

45) Por que pequenas mudanças nos dados podem gerar árvores de decisão muito diferentes?

Porque as árvores de decisão escolhem os splits de forma sequencial com base nos dados disponíveis. Pequenas alterações no conjunto de treino podem mudar a escolha dos primeiros splits, alterando toda a estrutura construída posteriormente. Isso torna as árvores modelos de alta variância e relativamente instáveis.

46) Qual a principal limitação de árvores isoladas?

A principal limitação de árvores isoladas é a alta variância, tornando o modelo instável e suscetível a overfitting. Pequenas mudanças nos dados de treino podem gerar árvores muito diferentes, prejudicando a capacidade de generalização.

47) Qual a diferença entre CART e ID3?

O ID3 utiliza entropia e ganho de informação como critério de divisão, foi desenvolvido apenas para problemas de classificação e pode gerar divisões multirramos. Já o CART (Classification and Regression Trees) foi desenvolvido tanto para classificação quanto para regressão, gera apenas divisões binárias e, em problemas de classificação, normalmente utiliza o índice Gini como critério de divisão.

48) Qual a principal evolução do C4.5 sobre o ID3?

O C4.5 é uma evolução do ID3. Portanto, mantém o foco em problemas de classificação, continua utilizando entropia e ganho de informação como critério de divisão e também permite divisões multirramos. Como evolução, adiciona suporte a atributos contínuos, tratamento de valores ausentes e mecanismos de poda para reduzir overfitting.

49) Qual o diferencial do C5.0?

O C5.0 é uma evolução do C4.5 focada em desempenho e eficiência. Ele gera árvores menores, mais rápidas, com menor consumo de memória e melhor capacidade de generalização.

Questionário (ref. Aulas 10 a 16)

Random Forest

50) O que é ensemble?

Ensemble é uma técnica que combina múltiplos modelos para produzir uma única predição mais robusta e estável. Esses modelos podem ser do mesmo tipo ou de tipos diferentes, incluindo algoritmos de machine learning e modelos estatísticos. A ideia é explorar a diversidade entre os modelos para reduzir erros individuais e melhorar a capacidade de generalização.

51) Qual o objetivo principal de combinar múltiplos modelos?

O principal objetivo é melhorar a capacidade de generalização e reduzir erros, tornando as previsões mais estáveis, robustas e precisas do que as de um único modelo isolado.

52) O que é bagging?

Bagging (Bootstrap Aggregating) é uma técnica de ensemble em que vários modelos são treinados de forma independente em diferentes amostras aleatórias do dataset. As previsões desses modelos são então combinadas por média ou votação, com o objetivo principal de reduzir a variância e aumentar a estabilidade do modelo.

53) O que significa dizer que o bootstrap é feito “com reposição” no RF?

Significa que, ao gerar a amostra bootstrap para treinar cada árvore, as linhas do dataset original são sorteadas aleatoriamente permitindo repetição. Assim, uma mesma linha pode aparecer várias vezes na amostra, enquanto outras podem não aparecer. Normalmente, cada árvore recebe um novo dataset bootstrap com cardinalidade semelhante à do conjunto original.

54) Além do bootstrap das amostras, qual outra forma de aleatoriedade existe no Random Forest?

Além da aleatoriedade das amostras (linhas), o Random Forest também introduz aleatoriedade nas variáveis (colunas/features). Em cada split da árvore, o algoritmo sorteia apenas um subconjunto aleatório das variáveis candidatas para decidir o melhor corte. Isso aumenta a diversidade entre as árvores e reduz a correlação entre elas.

55) Qual problema o bagging tenta reduzir?

O bagging tenta reduzir principalmente a variância do modelo. A ideia é combinar vários modelos treinados em amostras bootstrap diferentes para produzir previsões mais estáveis e menos sensíveis a ruídos ou variações aleatórias dos dados. Como cada árvore aprende padrões um pouco diferentes, suas previsões individuais podem divergir. Porém, ao combinar

Questionário (ref. Aulas 10 a 16)

as respostas por votação ou média, os erros aleatórios tendem a se compensar parcialmente, tornando o modelo final mais robusto e estável.

56) O que significa dizer que as árvores são independentes no RF?
Significa que cada árvore é treinada separadamente, utilizando diferentes amostras bootstrap e subconjuntos aleatórios de variáveis. Assim, cada árvore aprende padrões um pouco diferentes das demais, aumentando a diversidade do ensemble.

57) Como a combinação de múltiplas árvores no RF torna o modelo mais estável e menos sensível a variações aleatórias dos dados?
O Random Forest reduz variância combinando múltiplas árvores de decisão treinadas em diferentes amostras aleatórias dos dados e com diferentes subconjuntos de variáveis. Como cada árvore comete erros diferentes, a agregação das previsões por média ou votação tende a suavizar erros individuais e tornar o modelo mais estável e robusto.

Observação: essa ideia está relacionada ao conceito estatístico de que médias de múltiplas amostras tendem a ser mais estáveis e menos sensíveis a flutuações aleatórias do que observações isoladas.

58) Como é feita a predição final no Random Forest?
No Random Forest, a predição final é obtida combinando as previsões de várias árvores. Em classificação, normalmente utiliza-se votação majoritária entre as árvores. Em regressão, utiliza-se a média dos valores previstos pelas árvores.

59) O Random Forest corrige viés elevado?
Não necessariamente. O principal objetivo do Random Forest é reduzir variância e overfitting por meio da combinação de várias árvores. Em alguns casos ele pode melhorar o viés, mas sua principal contribuição está na redução da variância e no aumento da estabilidade do modelo.

Questionário (ref. Aulas 10 a 16)

Gradient Boosting / Boosting

60) Qual a principal ideia do Gradient Boosting?

A principal ideia do Gradient Boosting é construir árvores sequencialmente, de modo que cada nova árvore tente corrigir os erros cometidos pelas árvores anteriores.

61) Qual a diferença entre Random Forest e Gradient Boosting?

No Random Forest, as árvores são treinadas de forma independente e paralela, buscando reduzir variância por meio de bagging e aleatoriedade. Já no Gradient Boosting, as árvores são construídas sequencialmente, onde cada nova árvore tenta corrigir os erros das anteriores.

62) No Gradient Boosting, as árvores são independentes?

Não. No Gradient Boosting, cada árvore depende das anteriores, pois é treinada para corrigir os erros produzidos pelo conjunto já existente.

63) O que significa aprendizado sequencial?

Significa que os modelos são treinados um após o outro, de forma dependente. Cada novo modelo utiliza informações dos erros anteriores para tentar melhorar o desempenho do ensemble.

64) O que são resíduos no Gradient Boosting?

Resíduos são os erros do modelo, calculados como a diferença entre o valor real e o valor previsto ($\text{erro} = y_{\text{real}} - y_{\text{pred}}$). No Gradient Boosting, cada nova árvore é treinada para aprender e corrigir esses resíduos deixados pelo modelo anterior.

65) Por que cada árvore aprende os erros da anterior?

Porque, no Gradient Boosting, cada nova árvore é treinada sobre os resíduos (erros) deixados pelo modelo anterior, tentando corrigir as previsões que ainda ficaram incorretas.

66) Qual problema o boosting tenta reduzir principalmente?

O boosting tenta reduzir principalmente o viés do modelo, construindo sucessivamente modelos capazes de corrigir erros anteriores e aumentar a capacidade de aprendizado do ensemble.

67) O que significa atualização incremental do modelo?

Significa que o modelo é construído gradualmente, adicionando novas árvores ao ensemble de forma sequencial. Cada nova árvore realiza um pequeno ajuste sobre o modelo já existente.

68) Como a predição final é construída no GBT?

Questionário (ref. Aulas 10 a 16)

A predição final é construída pela soma das contribuições de todas as árvores do ensemble. Cada nova árvore adiciona uma correção incremental às previsões anteriores do modelo.

69) Qual o papel do learning_rate?

O learning_rate controla o tamanho da contribuição de cada nova árvore no Gradient Boosting. Valores menores fazem com que o modelo aprenda mais lentamente e de forma mais gradual, enquanto valores maiores tornam as correções mais agressivas.

70) O que acontece se o learning_rate for muito alto?

Se o learning_rate for muito alto, o modelo pode realizar correções excessivas a cada etapa, tornando o aprendizado instável e aumentando o risco de overfitting.

71) Qual a vantagem do Gradient Boosting em padrões complexos?

O Gradient Boosting possui grande capacidade de capturar padrões complexos e não lineares, pois constrói árvores sequencialmente, onde cada nova árvore aprende os erros deixados pelas anteriores. Isso permite reduzir gradualmente o viés do modelo e melhorar a precisão das previsões.

72) Por que boosting pode overfitar?

Porque o boosting tenta corrigir sucessivamente os erros do modelo. Se muitas árvores forem adicionadas ou se as correções forem muito agressivas, o modelo pode começar a capturar ruídos e padrões aleatórios do conjunto de treino.

73) Qual a diferença entre bagging e boosting?

No bagging, os modelos são treinados de forma independente e paralela, buscando reduzir variância. Já no boosting, os modelos são treinados sequencialmente, onde cada novo modelo tenta corrigir os erros dos anteriores, reduzindo principalmente viés.

74) Qual reduz mais variância?

O bagging, especialmente o Random Forest, é mais voltado para redução de variância.

75) Qual reduz mais viés?

O boosting é mais voltado para redução de viés.

76) O que é o AdaBoost e qual sua principal ideia?

O AdaBoost é um método de boosting que treina modelos sequencialmente aumentando o peso das observações classificadas incorretamente anteriormente, fazendo com que os próximos modelos foquem mais nos casos difíceis.

Questionário (ref. Aulas 10 a 16)

77) O que é o XGBoost e por que ele surgiu?

O XGBoost é uma evolução otimizada do Gradient Boosting criada para aumentar desempenho, velocidade e controle de overfitting, adicionando regularização e diversas otimizações computacionais.

78) O que é o LightGBM e qual seu foco principal?

O LightGBM é uma evolução do boosting focada principalmente em velocidade e eficiência computacional, especialmente para grandes volumes de dados..

79) O que é o CatBoost e qual seu principal diferencial??

O CatBoost é um método de boosting desenvolvido para lidar de forma eficiente com variáveis categóricas, reduzindo a necessidade de transformações extensas nos dados.

Questionário (ref. Aulas 10 a 16)

Perguntas comparativas/interpretação

- 80) Qual modelo tende a ter maior custo de predição: KNN ou árvore?
KNN, porque precisa calcular distâncias para diversos pontos do conjunto de treino a cada nova predição.
- 81) Qual modelo depende mais de escalonamento: KNN ou árvore?
KNN, porque depende diretamente de medidas de distância.
- 82) Qual modelo sofre mais com alta dimensionalidade?
KNN, porque a noção de distância perde poder discriminativo em espaços de alta dimensionalidade.
- 83) Qual modelo costuma ser mais interpretável?
Árvore de decisão isolada, porque suas regras podem ser visualizadas e interpretadas diretamente.
- 84) Por que ensembles costumam generalizar melhor?
Porque combinam múltiplos modelos, reduzindo o impacto de erros individuais e tornando as previsões mais estáveis e robustas.
- 85) Qual técnica de ensemble é utilizada para:
- a. reduzir variância?
Bagging, especialmente Random Forest.
 - b. reduzir viés aprendendo erros anteriores?
Boosting, como Gradient Boosting e XGBoost.
 - c. treinar árvores paralelas e independentes?
Bagging / Random Forest.
 - d. treinar árvores sequencias e dependentes?
Boosting / Gradient Boosting.

Questionário (ref. Aulas 10 a 16)

Redes Neurais

86) O que é um neurônio artificial?

*Um neurônio artificial é uma estrutura matemática inspirada no funcionamento simplificado dos neurônios biológicos. Ele recebe entradas, aplica pesos, realiza uma **combinação linear**, adiciona um bias e produz uma saída após aplicar uma função de ativação.*

87) Qual o papel dos pesos em um neurônio artificial?

Os pesos controlam a importância de cada entrada no cálculo realizado pelo neurônio. Durante o treinamento, esses pesos são ajustados para melhorar as previsões do modelo.

88) O que é o bias em um neurônio artificial?

O bias é um termo adicional somado ao cálculo do neurônio que permite deslocar a função de ativação com relação à origem do espaço vetorial, aumentando a flexibilidade do modelo.

89) Qual o papel da função de ativação?

A função de ativação transforma a saída linear do neurônio e permite que a rede capture relações não lineares e padrões complexos. Sem funções de ativação, mesmo redes com várias camadas perderiam a capacidade de representar não linearidades e se comportariam essencialmente como um único modelo linear.

90) Qual a diferença entre Sigmoid, Tanh e ReLU?

*A Sigmoid produz saídas entre **0 e 1**, a Tanh produz saídas entre **-1 e 1**, e a ReLU retorna zero para valores negativos e mantém valores positivos inalterados, variando, portando, entre 0 e ∞ . Atualmente, a ReLU é uma das mais utilizadas devido à simplicidade computacional e melhor comportamento no treinamento de redes profundas.*

91) Por que a ReLU se tornou tão popular?

Porque ela é computacionalmente simples, reduz problemas de gradientes muito pequenos e acelera o treinamento de redes neurais profundas.

92) O que é o Perceptron?

O Perceptron é um dos primeiros modelos de neurônio artificial, capaz de realizar classificações lineares simples por meio de uma combinação linear das entradas seguida de uma função de ativação.

93) Qual a principal limitação do Perceptron clássico?

O Perceptron clássico consegue resolver apenas problemas linearmente separáveis. Isso significa que ele conseguia separar os dados utilizando

Questionário (ref. Aulas 10 a 16)

uma reta, plano ou hiperplano, mas não conseguia resolver padrões não lineares clássicos, como o problema XOR.

94) O que é uma MLP (Multi Layer Perceptron)?

Uma MLP é uma rede neural composta por múltiplas camadas de neurônios artificiais, incluindo camada de entrada, camadas ocultas e camada de saída.

95) Por que uma MLP rasa consegue resolver problemas não lineares enquanto o Perceptron clássico não consegue?

O Perceptron clássico não possui camada oculta, sendo composto basicamente por um único neurônio artificial. Por isso, ele consegue criar apenas fronteiras de decisão lineares. Já a MLP rasa possui uma camada oculta contendo múltiplos neurônios e funções de ativação não lineares, permitindo combinar múltiplas transformações e representar padrões não lineares mais complexos.

96) O que representa a matriz de pesos W em uma rede neural?

A matriz de pesos armazena os pesos das conexões entre neurônios de diferentes camadas da rede.

97) Por que redes neurais utilizam operações matriciais?

Porque operações matriciais permitem representar e calcular simultaneamente múltiplas conexões entre neurônios de forma eficiente e paralelizável.

98) O que significa treinamento em redes neurais?

Significa ajustar iterativamente os pesos e bias da rede para reduzir o erro entre as previsões do modelo e os valores reais.

99) O que é propagação forward (forward propagation)?

É o processo em que os dados percorrem a rede da entrada até a saída, produzindo uma previsão.

100) O que é função de perda (loss function)?

É uma função matemática utilizada para medir o erro entre a previsão produzida pela rede e o valor real esperado.

101) O que é backpropagation?

É o algoritmo utilizado para propagar o erro da saída para as camadas anteriores da rede, permitindo calcular como os pesos devem ser ajustados.

102) O que significa dizer que redes neurais aprendem ajustando pesos?

Significa que o aprendizado ocorre modificando os pesos das conexões entre neurônios para reduzir gradualmente o erro do modelo.

Questionário (ref. Aulas 10 a 16)

103) Em uma MLP rasa com 3 atributos de entrada e 5 neurônios na camada oculta, qual será o shape da matriz de pesos W_1 ? Justifique.

O shape da matriz W_1 será (3×5) , pois a rede possui 3 atributos de entrada e 5 neurônios na camada oculta. Cada neurônio da camada oculta precisa receber um peso associado a cada atributo de entrada.

104) Cite cinco dos principais hiperparâmetros de uma MLP.

Alguns dos principais hiperparâmetros de uma MLP são: quantidade de camadas ocultas, quantidade de neurônios por camada, `learning_rate`, função de ativação e número de épocas (epochs). Esses hiperparâmetros controlam a arquitetura, a velocidade de aprendizado e a complexidade da rede neural.